

• Sportplatz

Documento maestro · Fily & Co

0	Cómo se usa este documento	8	Pagos
1	Qué es Sportplatz	9	Neo AI en producción
2	Usuarios y roles	10	Sistema de diseño
3	Las pantallas	11	Trampas ya documentadas
4	Reglas de negocio	12	Plan de construcción
5	Modelo de datos	13	Medición
6	Arquitectura	14	Decisiones abiertas
7	Cuentas y multi-tenancy	15	Anexos

Demo en vivo

sportplatz-eight.vercel.app

Sportplatz — Documento maestro

De la demo a la plataforma. Todo lo que hay que saber para construir Sportplatz como SaaS multi-tenant real: qué es, para quién, qué hace cada pantalla, cuáles son las reglas exactas, cómo se guardan los datos, cómo se cobra, cómo habla el bot y cómo se ve.

- **Demo en vivo:** <https://sportplatz-eight.vercel.app>
- **Código de referencia:** este repositorio (HTML/CSS/JS plano, sin build)
- **Estado:** demo terminada a nivel de diseño y comportamiento. La lógica funciona de verdad; lo que falta es backend, cuentas, dinero y WhatsApp real.
- **Fecha del documento:** 2026-08-20

0. Cómo se usa este documento

Está escrito para dos lectores y no hace falta leerlo entero para empezar:

SI ERES...	LEE	SÁLTATE
Dueño de producto / cliente	§1 a §4 y §12	§5 a §9
Dev o agencia que construye	§4 a §12	nada
Agente de código (Claude Code, Cursor)	§4, §5, §10, §11, §12	§1 a §3

Tres reglas que atraviesan todo el documento:

- 1. **La demo es la especificación.** Cuando este documento y el código de la demo no coincidan, gana el código: está probado con usuarios. Lo que aquí se marca como *NUEVO* es lo que la demo todavía no hace.
- 2. **No se rediseña.** El sistema visual está resuelto y documentado en `DESIGN.md`. Migrar de stack no es licencia para cambiar el aspecto.
- 3. **Si la agenda miente, el producto no vende.** Toda regla de disponibilidad, dinero y torneos se implementa exacta. Las reglas están en §4, numeradas, para poder citarlas en un ticket.

1. Qué es Sportplatz

Software de gestión para **complejos deportivos**: canchas sintéticas, pádel, tenis, voleibol playa y paintball. Un negocio que hoy se administra con un cuaderno, un grupo de WhatsApp y la memoria del que atiende.

1.1 El problema real

Un complejo de diez canchas mueve entre 40 y 90 reservas por semana. Hoy:

- Las reservas llegan por WhatsApp, a cualquier hora, y alguien tiene que contestar. Si contesta tarde, el cliente reserva en otro lado.
- El adelanto se pide por Nequi y se anota en el cuaderno. Nadie sabe con certeza cuánto se ha **cobrado** este mes, solo cuánto se ha **facturado**.
- Se pierden horas por doble reserva y por huecos que nadie llenó.
- Los torneos se organizan en una hoja de papel que se moja.

1.2 Qué resuelve

Para el dueño: una agenda que no se contradice, la plata separada entre facturado y cobrado, saldos por cobrar con nombre y teléfono, y un asistente de WhatsApp que atiende y aparta sin que nadie conteste.

Para el jugador: reservar en veinte segundos sin llamar, armar su equipo, inscribirse a torneos y ver su histórico.

1.3 El momento que vende

El visitante reserva una cancha como jugador, toca el conmutador, entra como dueño y ve **su propia reserva** en la agenda, en la hora exacta.

Veinte segundos, sin explicación. Todo lo demás de la plataforma existe para que ese momento sea posible y creíble. En la plataforma real ese momento se convierte en la promesa comercial: *lo que el jugador hace, el dueño lo ve al instante*.

1.4 Qué NO es

- No es un marketplace de canchas. El dueño no compite en una lista; la plataforma es suya y lleva su nombre y su logo.
- No es un CRM genérico.
- No es una app nativa. Es web, y se manda por WhatsApp como link.

2. Usuarios y roles

ROL	QUIÉN ES	QUÉ QUIERE	CÓMO ENTRA
Dueño	Paga la plataforma	Ver plata, ocupación y saldos	Correo y contraseña
Operador (NUEVO)	Quien atiende el mostrador	Crear reservas y cobrar, sin ver finanzas	Invitación del dueño
Jugador	El cliente del dueño	Reservar, equipo, torneos, histórico	Teléfono con código (OTP)
Fily admin (NUEVO)	Nosotros	Alta de complejos, soporte, facturación	Panel aparte

Decisiones de rol que se heredan de la demo:

- **El rol no es un filtro de permisos: son dos productos distintos sobre el mismo estado.** El dueño ve plata y ocupación; el jugador ve su próximo partido. Cambia la navegación entera, no los datos que se ocultan.
- En la demo el rol se cambia con un conmutador sin autenticación. En producción son sesiones distintas — pero **el conmutador se conserva en la demo comercial**, que sigue viviendo y es la herramienta de venta.
- Un mismo ser humano puede ser dueño de un complejo y jugador en otro. Los perfiles son independientes y se enlazan por teléfono.

3. Las pantallas

Trece vistas, más el onboarding. Nombres exactos como están en el producto.

3.1 Onboarding — cuatro preguntas

No da la bienvenida: **personaliza**. Con estas cuatro respuestas se genera el negocio entero.

1. **Nombre del complejo y WhatsApp**. El número no es un dato de formulario: es el número al que Neo mandaría las reservas, y la demo se lo enseña funcionando con su propio número.
2. **Qué deportes ofrece**. Selección múltiple entre los cinco.
3. **Cuántas canchas de cada uno**. Contador por deporte. Foto opcional por deporte, arrastrando el archivo.
4. **A qué precio la hora**. Precio por deporte, prellenado con el precio sugerido de mercado.

Al terminar: se crean las canchas, se genera la historia de arranque (§4.9) y se entra al Panel.

3.2 Lado del dueño — siete vistas

Panel — Facturado de hoy y de la semana. **Cobrado** del mes, que no es lo mismo que facturado y aquí se distingue. Saldos por cobrar. Ingresos diarios de los últimos 14 días. Ranking de qué cancha deja más plata. Próximas reservas y quién debe cuánto.

Canchas — La foto de la cancha del deporte ocupa todo el fondo y las tarjetas flotan encima en vidrio oscuro. El selector salta entre deportes y **cambia el color de toda la sección**. Cada cancha con nombre y precio editables, estado en vivo y la tira de horas del día.

Reservas — Agenda hora × cancha, 14 días navegables, filtro por deporte. Crear reserva con adelanto, registrar abonos, cancelar. Verde = pagada, ámbar = con adelanto, rojo = sin abonar. **WA** marca las que entraron por Neo. Scrollea en horizontal con la columna de horas y la cabecera fijas: aguanta 40 canchas.

Torneos — Crear, editar, sortear cruces, eliminar. 4, 8 o 16 equipos. Llaves donde los ganadores avanzan solos, calendario, tabla de posiciones, anotadores, valla menos vencida y podio. Se pueden abrir a inscripción.

Equipos — El directorio del complejo: clubes con escudo y récord, jugadores con ficha, y un salón de la fama con máximos anotadores y clubes más ganadores.

Neo AI — Conversación de WhatsApp donde el bot cotiza, explica el adelanto, aparta y confirma con código, mientras al lado se encienden las cinco cosas que hace por detrás.

Ajustes — Nombre y logo del negocio, **horario de atención**, renombrar y tarifar cada cancha, fotos, reiniciar.

3.3 Lado del jugador — seis vistas

Inicio — Su próximo partido, sus reservas, cómo va su equipo, qué canchas están libres en este momento.

Reservar — Deporte, día, cancha y hora en una sola pantalla, con la parrilla de horas libres. Elige si aparta con adelanto, paga completo o paga en cancha.

Mi equipo — Crearlo, ponerle escudo y color, armar la plantilla.

Torneos — Ver los que hay, meter su equipo en un cupo libre, seguir la llave.

Mi histórico — Partidos, anotaciones, títulos, equipos y en qué canchas juega más. Nada de esto se guarda: se deriva (§4.7).

Neo AI — El mismo asistente, dentro de la app.

3.4 El conmutador

Abajo a la izquierda: *Viendo como dueño / jugador*. Es lo que convierte esto en un demo y no en una captura.

Se conserva en la demo comercial; en la plataforma real desaparece y lo reemplaza la sesión.

4. Reglas de negocio

Numeradas para poder citarlas. Todas están implementadas en la demo y hay que implementarlas igual.

4.1 Franjas y horario

- **R1** La unidad de reserva es **una hora exacta**. `start = '18:00'`, `end = start + 1h`. No hay medias horas ni bloques de 90 minutos. (*Extender a duración variable es cambio de producto, no de implementación: ver §14.*)
- **R2** El horario de atención es del negocio: `openHour` (por defecto 6) y `closeHour` (por defecto 23). Las franjas del día van de `openHour` a `closeHour - 1`.
- **R3** Cambiar el horario **no borra** reservas que queden fuera. Se muestran igual y se avisa.

4.2 Disponibilidad

- **R4** Una cancha tiene como máximo **una reserva por franja**. La clave es `(courtId, date, start)`. En la demo se valida en cliente; en producción es una **restricción única en base de datos**, no una comprobación optimista.
- **R5** Antes de confirmar se vuelve a comprobar. Si en el intervalo alguien tomó la hora, se rechaza y se ofrecen las horas libres restantes. Neo ya lo hace: "*Uy, alguien acaba de tomar esa hora mientras hablábamos.*"
- **R6** Horas libres de una cancha en un día = franjas del horario menos las ocupadas. Las horas ya pasadas de hoy no se ofrecen.
- **R7** Una cancha con `active: false` no se ofrece ni cuenta para ocupación.

4.3 Dinero

- **R8** `total` = precio de la cancha en el momento de reservar. **Se congela en la reserva**: cambiar la tarifa mañana no reescribe el pasado.
- **R9** `deposit` = lo efectivamente recibido. Empieza en 0, en el 50% o en el total. Nunca supera a `total`.
- **R10** El **adelanto sugerido** es el 50% del precio redondeado a miles hacia el múltiplo más cercano: `round(price * 0.5 / 1000) * 1000`. Ejemplo: cancha de \$120.000 → adelanto \$60.000; de \$25.000 → \$13.000.
- **R11** **Saldo** = `total - deposit`. Cero significa pagada.
- **R12** **Facturado** en un rango = suma de `total` de las reservas cuya `date` cae en el rango.
- **R13** **Cobrado** en un rango = suma de `deposit` de esas reservas. Facturado y cobrado **nunca se presentan como el mismo número**. Esta distinción es media venta.
- **R14** **Por cobrar** = suma de saldos de reservas de hoy en adelante.
- **R15** Un abono suma a `deposit` y nunca lo pasa de `total`.
- **R16** Moneda: peso colombiano, sin decimales. Formato `$120.000` (`es-CO`). En tarjetas de métrica se abrevia: `$1.2M`, `$340k`.

4.4 Estados y origen de la reserva

- **R17** `status`: `'confirmada'` para hoy y el futuro, `'jugada'` para el pasado. (*NUEVO en producción: `'cancelada'` y `'no_show'` — ver R19.*)
- **R18** `source`: de dónde entró.
- `'mostrador'` — la creó el dueño u operador.
- `'app'` — la creó el jugador desde Reservar.
- `'neo'` — hablando con Neo dentro de la app.
- `'bot'` — Neo por WhatsApp. En la agenda se marca con `WA`. Las cuatro son la misma entidad. La vista de automatización suma `neo` y `bot`; la agenda las distingue.
- **R19** Cancelar en la demo **borra** la reserva. En producción **no se borra**: pasa a `'cancelada'`, conserva quién y cuándo canceló, y libera la franja. Un negocio necesita saber qué se cayó.
- **R20** (*NUEVO*) Política de cancelación por negocio: hasta N horas antes el adelanto se devuelve o se acredita. Por defecto: 24 horas, acreditado.

4.5 Semáforo de la agenda

- **R21** Verde = saldo 0. Ámbar = hay adelanto pero queda saldo. Rojo = sin abonar nada. El color nunca es la única señal: siempre lleva su etiqueta.

4.6 Torneos

- **R22** Tamaños: **4, 8 o 16 equipos**. Eliminación directa.
- **R23** Rondas por tamaño: 16 → octavos, cuartos, semifinal, final. 8 → cuartos, semifinal, final. 4 → semifinal, final. Siempre hay partido de **tercer puesto**, alimentado por los perdedores de semifinal.
- **R24** Los ganadores **avanzan solos**: al marcar un resultado, la llave se resuelve hacia adelante. Un empate no produce ganador (eliminación directa).
- **R25** Tabla: $PTS = G*3 + E$. Orden: puntos, diferencia, menos recibidos, más anotados.
- **R26** Valla menos vencida: entre los que ya jugaron, menos goles en contra; desempata más partidos jugados, luego más puntos.
- **R27** Podio: oro = ganador de la final, plata = perdedor de la final, bronce = ganador del tercer puesto.
- **R28** Un torneo `openSignup` acepta que un jugador meta **su equipo** en el primer cupo libre. El cupo pasa a `taken` y no se puede reasignar sin el dueño.
- **R29** Volver a sortear (`reseed`) baraja los cruces de primera ronda, borra los marcadores de esa ronda y resuelve la llave de nuevo. **Solo el dueño**, y solo si no ha empezado.
- **R30** Los equipos del torneo **apuntan a equipos reales** por `teamId`. Un equipo que juega tres torneos es el mismo equipo las tres veces: eso es lo único que hace posible el histórico.

4.7 Histórico derivado — la regla que más se rompe

- **R31** **El histórico NO se guarda. Se deriva de los partidos.** Récord de un equipo, goles de un jugador, títulos, racha, salón de la fama: todo sale de `matches` y `scorers` en tiempo de lectura.

- **R32** Prohibido crear columnas acumuladas (`goles_totales` , `partidos_ganados`). Un marcador vive en un solo sitio. Guardar totales obliga a mantenerlos en sincronía cada vez que alguien corrige un resultado, y tarde o temprano se desincronizan.
- **R33** Si el cálculo pesa, se resuelve con **vistas materializadas o caché con invalidación por torneo**, nunca duplicando el dato en la tabla.
- **R34** Racha = los cinco resultados más recientes, del más viejo al más nuevo, para leerla como línea de tiempo de izquierda a derecha.

4.8 Equipos y jugadores

- **R35** Un jugador pertenece a varios equipos. Un equipo tiene un capitán (`captainId`), que está siempre dentro de la plantilla.
- **R36** El tamaño sugerido de plantilla depende del deporte: fútbol 7, paintball 5, voleibol 4, pádel 2, tenis 2.
- **R37** Posiciones por deporte: fútbol (Arquero, Defensa, Volante, Delantero), pádel (Drive, Revés), tenis (Fondo, Red), voleibol (Armador, Punta, Central, Líbero), paintball (Frontal, Medio, Back).
- **R38** Un equipo sin escudo **no muestra un cuadro gris**: muestra su inicial sobre su color, con el corte de escudo real.

4.9 Datos de arranque (*solo demo*)

- **R39** La semilla genera **75 días de historia y 14 hacia adelante**, siempre **relativos a hoy**. Nunca fechas fijas en el código: la demo no puede verse caducada.
- **R40** Carga por día de la semana: sábado 0,75, viernes 0,70, domingo 0,55, entre semana 0,38. Por hora: 17–22 pleno, 10–15 al 35%, resto al 12%.
- **R41** El pasado va todo pagado. El futuro: 42% pagado, 43% con adelanto, 15% sin abonar. 62% entró por bot. 6 de cada 10 las hace un jugador con cuenta, para que el histórico del jugador nazca con contenido.
- **R42** La generación recorre cada terna (cancha, día, hora) **una sola vez**: un duplicado es imposible por construcción. La versión que comprobaba con `some()` sobre un array creciente tardaba 10,5 s con 40 canchas.

4.10 Neo AI — reglas de conversación

- **R43** Neo **no simula**: lee la agenda real y escribe en ella. La reserva que crea hablando es **el mismo objeto** que crea el formulario.
- **R44** La conversación tiene un solo objetivo: llenar cuatro huecos — **deporte, día, cancha, hora** — y confirmar. Lo demás es acompañamiento.
- **R45** Neo nunca promete una hora ocupada. Lee disponibilidad de verdad antes de ofrecer.
- **R46** Entiende cómo se habla aquí: "*mañana a las 7*" es de noche, "*el sábado*" es el sábado que viene, "*la sintética*" es fútbol. La ambigüedad a. m./p. m. se resuelve hacia la tarde y **se avisa**.
- **R47** Tres formas de pago en la conversación: `adelanto` (50%), `completo`, `cancha` (paga al llegar, `deposit: 0`).

- **R48** Cada reserva sale con **código corto** SP-XXXX (últimos cuatro del id, en mayúsculas). Sirve para cancelar: *"cancelar SP-3F9A"*.
- **R49** Tras tres mensajes seguidos sin entender, Neo ofrece opciones rápidas en vez de seguir preguntando abierto.

5. Modelo de datos

5.1 El estado de hoy (demo)

Un único objeto `S`, plano, en memoria, persistido en IndexedDB:

```
S = {
  business: {
    name, whatsapp, sports: ['futbol','padel'],
    openHour: 6, closeHour: 23, logo,
    theme: { accent: 'esmeralda', fondo: 'estadio' }
  },
  courts: [{ id, sport, n, name, price, image, photo, active }],
  bookings: [{ id, courtId, date:'2026-08-20', start:'18:00', end:'19:00',
    playerId, customer, phone, total, deposit,
    source:'mostrador'|'app'|'neo'|'bot',
    status:'confirmada'|'jugada', note }],
  players: [{ id, name, phone, position, avatar, color, since }],
  teams: [{ id, name, sport, color, crest, captainId, roster:[playerId], created }],
  tournaments: [{
    id, name, sport, size:4|8|16, startDate, openSignup,
    teams: [{ id, teamId, name, color, taken }],
    matches: [{ id, round, idx, teamA, teamB, scoreA, scoreB,
      date, time, court, played, isThird }],
    scorers: [{ id, team, playerId, player, goals }]
  }],
  photos: { sports: { futbol: dataURL } },
  session: { role: 'owner'|'player', playerId }
}
```

Por qué está así y por qué cambia. La app entera lee el estado de forma **síncrona** (`S.bookings.filter(...)` aparece en cada vista). Eso hizo posible una demo sin backend y sin latencia. En producción esa forma no sirve: hace falta consultar entre negocios, dos personas editan a la vez, y el documento crece sin techo.

5.2 El esquema de producción (Postgres)

Multi-tenant por `complejo_id`, con RLS en todas las tablas. Sin columnas acumuladas (R31–R33).

```
-- — Tenencia —————
create table complejos (
  id          uuid primary key default gen_random_uuid(),
  slug        text unique not null,          -- para la URL pública
  nombre      text not null,
  whatsapp    text,
  logo_url    text,
  hora_abre   smallint not null default 6,
  hora_cierra smallint not null default 23,
  moneda      text not null default 'COP',
  zona_horaria text not null default 'America/Bogota',
  tema        jsonb not null default '{}',
  plan        text not null default 'trial',
  creado_en   timestampz not null default now()
);
```

```

-- Un ser humano. Puede ser dueño de un complejo y jugador en otro.
create table personas (
  id          uuid primary key references auth.users(id) on delete cascade,
  nombre      text not null,
  telefono    text unique,
  email       text,
  avatar_url  text,
  creado_en   timestampz not null default now()
);

-- Quién puede qué, dentro de qué complejo.
create table membresias (
  complejo_id uuid not null references complejos(id) on delete cascade,
  persona_id  uuid not null references personas(id) on delete cascade,
  rol         text not null check (rol in ('dueno','operador','jugador')),
  creado_en   timestampz not null default now(),
  primary key (complejo_id, persona_id, rol)
);

-- — Recursos —————
create table canchas (
  id          uuid primary key default gen_random_uuid(),
  complejo_id uuid not null references complejos(id) on delete cascade,
  deporte     text not null check (deporte in
    ('futbol','padel','tenis','voleibol','paintball')),
  numero      smallint not null,
  nombre      text not null,
  precio      integer not null check (precio >= 0),    -- COP, sin decimales
  foto_url    text,
  activa      boolean not null default true,
  orden       smallint not null default 0
);
create index on canchas (complejo_id, deporte);

-- — Agenda —————
create table reservas (
  id          uuid primary key default gen_random_uuid(),
  complejo_id uuid not null references complejos(id) on delete cascade,
  cancha_id   uuid not null references canchas(id) on delete cascade,
  fecha       date not null,
  inicio      time not null,                          -- siempre en punto (R1)
  fin         time not null,
  persona_id  uuid references personas(id),            -- null: cliente sin cuenta
  cliente     text not null,                          -- nombre congelado
  telefono    text,
  total       integer not null check (total >= 0),      -- congelado (R8)
  abonado     integer not null default 0 check (abonado >= 0 and abonado <= total),
  origen      text not null default 'mostrador'
    check (origen in ('mostrador','app','neo','bot')),
  estado      text not null default 'confirmada'
    check (estado in ('confirmada','jugada','cancelada','no_show')),
  nota        text default '',
  codigo      text generated always as
    ('SP-' || upper(right(id::text, 4))) stored,        -- R48
  cancelada_en timestampz,
  cancelada_por uuid references personas(id),
  creado_en   timestampz not null default now()
);

```

```

-- R4: la no-doble-reserva es una restricción, no una comprobación.
create unique index reservas_franja_unica
  on reservas (cancha_id, fecha, inicio)
  where estado in ('confirmada','jugada');

create index on reservas (complejo_id, fecha);
create index on reservas (persona_id, fecha desc);

-- Cada movimiento de plata, no un contador. `abonado` se mantiene por trigger
-- sobre esta tabla: es la única acumulación permitida, y es reconstruible.
create table pagos (
  id          uuid primary key default gen_random_uuid(),
  complejo_id uuid not null references complejos(id) on delete cascade,
  reserva_id  uuid not null references reservas(id) on delete cascade,
  monto       integer not null check (monto <> 0),    -- negativo = devolución
  medio       text not null check (medio in
    ('efectivo','transferencia','pasarela','cortesia')),
  referencia  text,                                  -- id de la pasarela
  registrado_por uuid references personas(id),
  creado_en   timestampz not null default now()
);

-- — Comunidad —————
create table equipos (
  id          uuid primary key default gen_random_uuid(),
  complejo_id uuid not null references complejos(id) on delete cascade,
  nombre      text not null,
  deporte     text not null,
  color       text not null,
  escudo_url  text,
  capitan_id  uuid references personas(id),
  creado_en   timestampz not null default now()
);

create table plantillas (
  equipo_id  uuid not null references equipos(id) on delete cascade,
  persona_id uuid not null references personas(id) on delete cascade,
  posicion   text,
  desde      date not null default current_date,
  primary key (equipo_id, persona_id)
);

create table torneos (
  id          uuid primary key default gen_random_uuid(),
  complejo_id uuid not null references complejos(id) on delete cascade,
  nombre      text not null,
  deporte     text not null,
  tamano      smallint not null check (tamano in (4,8,16)),  -- R22
  fecha_inicio date not null,
  inscripcion_abierta boolean not null default false,
  creado_en   timestampz not null default now()
);

create table torneo_cupos (
  id          uuid primary key default gen_random_uuid(),
  torneo_id   uuid not null references torneos(id) on delete cascade,
  posicion    smallint not null,
  equipo_id   uuid references equipos(id) on delete set null,
  nombre      text not null,
  color       text not null,

```

```

    unique (torneo_id, posicion)
);

create table partidos (
  id          uuid primary key default gen_random_uuid(),
  torneo_id   uuid not null references torneos(id) on delete cascade,
  ronda       text not null check (ronda in
    ('octavos','cuartos','semis','final','tercero')),
  idx         smallint not null,
  cupo_a      uuid references torneo_cupos(id),
  cupo_b      uuid references torneo_cupos(id),
  marcador_a  smallint,
  marcador_b  smallint,
  fecha       date,
  hora        time,
  cancha_id   uuid references canchas(id),
  jugado      boolean not null default false,
  unique (torneo_id, ronda, idx)
);

-- Anotaciones: el hecho, no el total (R31).
create table anotaciones (
  id          uuid primary key default gen_random_uuid(),
  partido_id  uuid not null references partidos(id) on delete cascade,
  cupo_id     uuid not null references torneo_cupos(id) on delete cascade,
  persona_id  uuid references personas(id),
  nombre      text not null,          -- por si no tiene cuenta
  goles       smallint not null default 1 check (goles > 0)
);

```

5.3 Cálculos derivados

Se implementan como **vistas** o funciones puras sobre el modelo. Portar tal cual desde `app/core/calc.js`, `tournament.js` y `teams.js`:

CÁLCULO	REGLA	ORIGEN EN LA DEMO
Facturado / Cobrado	R12, R13	revenueBetween, collectedBetween
Por cobrar	R14	pendingTotal
Ocupación	frangas usadas / frangas posibles	occupancy
Ranking por cancha	R12 agrupado	revenueByCourt
Serie diaria (14 d)	R12 por día	dailySeries
Horas libres	R6	freeHours
Tabla del torneo	R25	standings
Valla menos vencida	R26	cleanSheets
Podio	R27	podium
Récord de equipo	R31	teamRecord
Ficha de jugador	R31	playerRecord
Salón de la fama	R31	topScorersAllTime, teamLeaderboard

5.4 Seguridad de fila (RLS)

Es lo único que separa los datos de un negocio de los de otro. Si esto está mal, todo lo demás da igual.

```
alter table reservas enable row level security;

-- Pertenencia al complejo: la función que usan todas las políticas.
create or replace function es_del_complejo(c uuid, roles text[] default null)
returns boolean language sql stable security definer set search_path = '' as $$
  select exists (
    select 1 from public.membresias m
    where m.complejo_id = c
      and m.persona_id = auth.uid()
      and (roles is null or m.rol = any(roles))
  );
$$;

-- Dueño y operador ven toda la agenda del complejo.
create policy "agenda del complejo" on reservas for select to authenticated
  using (es_del_complejo(complejo_id, array['dueno','operador']));

-- El jugador ve solo lo suyo.
create policy "mis reservas" on reservas for select to authenticated
  using (persona_id = auth.uid());

-- El jugador crea reservas a su nombre, en un complejo donde está inscrito.
create policy "reservar" on reservas for insert to authenticated
```



```
with check (persona_id = auth.uid()
            and es_del_complejo(complejo_id, array['jugador','dueno','operador']));
```

Reglas que se repiten en cada tabla:

- **Nunca una política para** `anon`. Quien no ha entrado no lee ni escribe.
- **El jugador nunca ve el dinero agregado del complejo.** Ve su propia reserva y su propio saldo.
- **El** `service_role` **no aparece en el navegador.** Salta RLS por completo y vive solo en el servidor.

6. Arquitectura

6.1 Stack

CAPA	ELECCIÓN	POR QUÉ
Framework	Next.js (App Router) + TypeScript	Server Components para lo que es lectura pesada; Server Actions para escribir
Estilos	Tailwind con los tokens de <code>themes/momentum.css</code>	Los tokens ya son la fuente de verdad; se mapean a <code>tailwind.config</code>
Componentes	shadcn/ui retematizado	Base para componer, nunca el look final
Movimiento	motion (Framer Motion)	Mismas curvas y duraciones de <code>DESIGN.md</code>
Datos	Postgres (Supabase o Neon)	RLS, realtime, y ya hay migración escrita
Auth	Supabase Auth	Correo/contraseña para dueño, OTP por teléfono para jugador
Archivos	Supabase Storage o Vercel Blob	Fotos de cancha, logos, escudos
Hosting	Vercel	Fluid Compute; funciones Node por defecto
Notificaciones	WhatsApp Cloud API	\$9

Al provisionar cada servicio externo (base de datos, pagos, correo, observabilidad), pasar por el Marketplace de Vercel antes de escribir código de SDK. Integración real desde el primer día, no mock.

6.2 Renderizado y datos

- **Lectura del dueño** (Panel, Reservas, Torneos): Server Components que consultan directo. Nada de `useEffect` + `fetch`.
- **Escritura:** Server Actions con revalidación por etiqueta (`revalidateTag('agenda:'+complejoId)`).
- **Realtime:** suscripción a `reservas` filtrada por `complejo_id`. Es lo que hace que el dueño vea caer la reserva del jugador sin recargar — el momento que vende (§1.3), ahora de verdad.
- **Optimista en la agenda:** crear una reserva pinta la celda de inmediato y revierte si la restricción única la rechaza (R4).

6.3 Estructura de rutas

```
app/  
  (marketing)/           la landing y el demo público  
  (demo)/demo/           la demo actual, sin cuenta, tal como está hoy  
  (app)/[complejo]/      la plataforma real, multi-tenant por slug  
    panel/ canchas/ reservas/ torneos/ equipos/ neo/ ajustes/  
    j/                   el lado del jugador
```

```

    inicio/ reservar/ equipo/ torneos/ historial/ neo/
api/
    wa/webhook/          entrada de WhatsApp Cloud API
    pagos/webhook/       confirmación de la pasarela

```

- El **tenant sale de la URL** (`/[complejo]`), se resuelve en un layout y se valida contra `membresias`. Nunca se confía en un `complejo_id` que venga del cliente.
- La **demo pública sobrevive** en `/demo`: es la herramienta de venta y se mantiene sin cuenta, con el conmutador de rol y todo su estado en el navegador.

6.4 Lo que se copia y lo que se reescribe

ARCHIVO ACTUAL	DESTINO	TRABAJO
<code>app/core/util.js</code>	<code>lib/util.ts</code>	Copiar y tipar
<code>app/core/sports.js</code>	<code>lib/sports.ts</code>	Copiar tal cual (SVG incluidos)
<code>app/core/calc.js</code>	<code>lib/queries/*.ts</code>	Reescribir contra SQL, misma firma
<code>app/core/tournament.js</code>	<code>lib/tournament.ts</code>	Copiar; puro
<code>app/core/teams.js</code>	<code>lib/records.ts</code>	Copiar; puro (R31)
<code>app/core/neo.js</code>	<code>lib/neo/</code>	Copiar el motor; cambiar lectura/escritura por consultas
<code>app/core/seed.js</code>	<code>lib/seed.ts</code>	Solo para la demo y para datos de prueba
<code>app/core/store.js</code>	—	Se tira. Lo reemplazan las consultas
<code>app/ui/shell.js</code>	<code>app/(app)/layout.tsx</code>	Layout con navegación por rol
<code>app/views/*.js</code>	un componente por vista	<code>viewX(main)</code> → <code><X/></code>
<code>app/base.css</code> + <code>premium.css</code>	globals + componentes	Migrar; no rediseñar
<code>themes/momentum.css</code>	<code>app/globals.css</code> + <code>tailwind.config</code>	Fuente de verdad del diseño

`core/` no toca el DOM. Por eso se copia: fue partido por esa costura a propósito.

7. Cuentas y multi-tenancy

7.1 Cómo entra cada quien

- **Dueño:** correo y contraseña. Al registrarse crea su complejo y pasa por el onboarding de cuatro preguntas (§3.1), que ahora escribe en base de datos.
- **Operador:** el dueño lo invita por correo o por link. Rol `operador`.
- **Jugador: teléfono + código por WhatsApp.** Es el dato que ya tiene el negocio y el que usa para reservar. Sin contraseña.
- **El teléfono es la llave de unión:** si el dueño creó reservas a nombre de un teléfono y esa persona luego abre cuenta, sus reservas anteriores se le atribuyen automáticamente.

7.2 Reglas de aislamiento

1. Toda tabla lleva `complejo_id` y RLS. Sin excepción.
2. El tenant se resuelve en el servidor, del slug de la URL, contra `membresias`. Nunca de un campo del formulario.
3. Un jugador ve su reserva, no la agenda. Un operador ve la agenda, no el Panel financiero.
4. `service_role` solo en funciones de servidor: webhooks, cron, migraciones.

7.3 Migración del demo a la cuenta

Cuando alguien que jugó con la demo se registra, **su complejo de demo se sube**: nadie pierde el trabajo por registrarse. Ya está implementado en `store.js` contra Supabase y se conserva el comportamiento.

8. Pagos

8.1 Qué se cobra y a quién

Dos flujos distintos que no se mezclan:

A. El jugador le paga al complejo (el adelanto de la reserva). **B. El complejo nos paga a nosotros** (la suscripción a la plataforma).

8.2 A · Adelanto de reserva

- Mercado objetivo: Colombia. Los medios que la gente usa son **Nequi, Bancolombia, PSE y tarjeta**.
- Candidatos: **Wompi** (Bancolombia; Nequi y PSE nativos, comisión clara), **Mercado Pago** (mejor conversión en móvil, más caro), **ePayco/PayU** (más medios, integración más áspera). Recomendación: **Wompi primero**, con la capa de pago detrás de una interfaz propia para poder cambiar.
- Reglas:
 - **P1** Se cobra el adelanto de R10 salvo que el jugador elija pagar completo o pagar en cancha.
 - **P2** La reserva se crea **antes** del pago, en estado `confirmada` con `abonado = 0`, y con la franja ya bloqueada por 15 minutos. Si el pago no llega, se libera. Bloquear después del pago produce cobros sin cancha.
 - **P3** El webhook de la pasarela es la **única** fuente de verdad del pago. El redirect del navegador no confirma nada.
 - **P4** Cada confirmación inserta una fila en `pagos` con la referencia de la pasarela. Idempotente por referencia: un webhook repetido no cobra dos veces.
 - **P5** Devolución = fila en `pagos` con monto negativo. Nunca se edita un pago existente.
 - **P6** El dueño puede registrar pagos en efectivo desde el mostrador; van a la misma tabla con `medio: 'efectivo'`.
- **Liquidación**: el dinero entra a la cuenta del complejo, no a la nuestra. Nosotros no somos intermediarios de fondos — decisión deliberada: serlo cambia el régimen regulatorio del negocio.

8.3 B · Suscripción

Cobro mensual por complejo, escalado por número de canchas. Prueba de 14 días sin tarjeta. El estado vive en `complejos.plan`. Cuando vence: la plataforma pasa a **solo lectura**, nunca se borran datos y la agenda se puede seguir consultando. Un negocio que no puede ver su agenda de mañana nos denuncia, con razón.

9. Neo AI en producción

9.1 Qué es hoy

Un motor determinista en español (`app/core/neo.js` , 570 líneas) que llena cuatro huecos y lee la agenda real. **No hay modelo de lenguaje detrás, y para el 80% de las conversaciones no hace falta:** la conversación tiene un solo objetivo. Lo que sí exige es entender cómo habla la gente aquí (R46).

9.2 Qué se agrega

Canal real: WhatsApp Cloud API (Meta).

1. Número de WhatsApp Business por complejo, o número compartido con identificación del negocio en el primer mensaje.
2. Webhook en `/api/wa/webhook` que verifica la firma de Meta, resuelve el complejo por el número de destino, y pasa el texto al motor.
3. **Ventana de 24 horas:** fuera de ella solo se puede escribir con *plantillas aprobadas*. Hacen falta al menos tres, aprobadas antes del lanzamiento: - `recordatorio_reserva` — 3 h antes del partido. - `saldo_pendiente` — el día del partido si queda saldo. - `cupo_liberado` — cuando se cae una reserva de una hora pico.
4. **Respuesta a lo no previsto:** si el motor determinista no entiende tras dos intentos (R49), se escala a un LLM con la agenda como contexto y con herramientas acotadas (`consultar_disponibilidad` , `crear_reserva` , `cancelar_reserva`). El LLM **nunca escribe directo en la base**: llama a las mismas funciones que el motor, que son las que validan R4 y R5.
5. **Traspaso a humano:** palabra clave o tres fallos seguidos y la conversación se marca para que el dueño la vea en la vista de Neo AI.

9.3 Reglas que no cambian al pasar a WhatsApp

- La reserva que crea Neo es **el mismo registro** que crea el mostrador (R43). Solo cambia `origen`.
- Neo nunca ofrece una hora que no está libre (R45), y revalida antes de cerrar (R5).
- Todo cierre entrega el código `SP-XXXX` (R48).
- Neo **no negocia precios ni inventa promociones**. Si no sabe, ofrece hablar con el dueño.

10. Sistema de diseño

El detalle completo está en `DESIGN.md` (26 KB, con el porqué de cada valor) y los valores en `themes/momentum.css`. Esto es lo que no se negocia al migrar.

10.1 Color

- Acento **verde esmeralda** `#047857`: 5.48:1 sobre blanco, así que el mismo color sirve de relleno **y** de texto, y encima lleva blanco. Sobre superficie oscura sube a `#34d399` (10.1:1 sobre negro).
- El verde también significa "pagado". No es un descuido: lo que separa marca de estado no es el tono sino **la forma** — la marca siempre es relleno sólido (botón, pastilla), el estado siempre es tinte de fondo con etiqueta al lado. Nunca compiten por el mismo sitio.
- **Canchas es la única sección que se retinta**, con el color real de cada deporte. El retinte va por tokens, nunca por componente.
- Colores de dato separados en **tono, no en luminancia**: se distinguen en escala de grises y para un daltónico. El rojo va al final del orden porque en esta interfaz significa "sin abonar".
- **Ningún color literal fuera del archivo de tokens**, salvo capas de luz y de velo (`rgba` blancos y negros). Si un color tiene nombre, es un token.

10.2 Tipografía

Una sola familia con **eje de ancho**: Archivo (wdth 62–125, wght 100–900). De ahí salen dos voces que no se parecen — el titular ancho y muy pesado de gráfico deportivo, y el texto normal — sin cargar una segunda fuente. IBM Plex Mono **solo** para dato técnico: horas, códigos.

El tracking es específico del tamaño. No hay un `letter-spacing` para todo:

PASO	TAMAÑO	INTERLÍNEA	TRACKING	PESO
display	<code>clamp(2.375rem, 4.3vw, 3.625rem)</code>	0.96	-0.038em	800
h1	<code>clamp(1.625rem, 2.5vw, 2.125rem)</code>	1.05	-0.03em	800
h2	1.125rem	1.28	-0.02em	700
h3	0.9375rem	1.35	-0.012em	700
body	0.9375rem	1.6	-0.004em	—
sm	0.8125rem	1.5	0	—
xs	0.75rem	1.42	+0.004em	—
micro	0.6875rem	1.35	+0.085em	—

Números siempre `tabular-nums lining-nums`.

10.3 Forma, elevación y material

- Radios: 8 / 12 / 16 / 20 / 26 px y pastilla. **Uno por rol, no uno por componente.**
- Cada nivel de elevación son **dos sombras**: una de contacto, corta y opaca, y una ambiental, larga y difusa. Nunca una sola sombra gigante.
- El rail es **negro mate dibujado con gradientes**, no una foto de metal, y lleva grano finísimo: sin él un negro grande forma bandas y se lee a plástico.
- Las tarjetas sobre foto son **vidrio oscuro** (`rgba(8,13,24,.60)` + `blur(20px) saturate(160%)`): se leen apoyadas sobre la foto en vez de taparla.

10.4 Fondo

La noche de estadio en toda la app; la cancha a sangre **solo** en Canchas. Por eso funciona: cuando el fondo cambia, cambia porque estás mirando otro deporte, no porque cambiaste de pestaña.

10.5 Movimiento

```
--ease-out      cubic-bezier(.23, 1, .32, 1)      entradas, hover, la mayoría
--ease-in-out   cubic-bezier(.77, 0, .175, 1)     movimiento en pantalla
--ease-drawer   cubic-bezier(.32, .72, 0, 1)       toast, hojas
--ease-pop      cubic-bezier(.34, 1.12, .48, 1)    confirmaciones y podio
```

Duraciones: 140 ms presión · 180 ms hover · 220 ms elevación · 320 ms modal.

Sin excepción: **nunca ease-in en UI**, nunca `transition: all`, solo `transform` y `opacity`, todo lo pulsable baja a `scale(.97)` en `:active`, nada entra desde `scale(0)`, hover detrás de `@media (hover: hover) and (pointer: fine)`, stagger de 38–55 ms en listas.

Al pasar a React: **motion**, manteniendo exactamente estas curvas y duraciones.

10.6 Accesibilidad, ya resuelta — no romperla

Modal con foco atrapado que vuelve al disparador y fondo `inert`. Jerarquía de encabezados sin saltos (las tarjetas de rejilla son `h2`). Salto al contenido. Objetivos táctiles de 34 px mínimo. `role="log"` + `aria-live` en Neo, `role="status"` en el toast. `prefers-reduced-motion`, `prefers-reduced-transparency` y `prefers-contrast` respetados.

10.7 Anti-patronos que este sistema rechaza

Texto con degradado · una sola tipografía sin jerarquía real · gradientes morado→azul de marca · cards dentro de cards · texto gris de bajo contraste sobre color · icono en cuadrado sobre cada heading · emojis como iconografía · sombras difusas sin lógica de elevación · un `border-radius` por componente · retículas decorativas de fondo · borde lateral grueso de color en tarjetas.

11. Trampas ya documentadas

Volver a caer en estas es retroceder. Están explicadas en `DESIGN.md` y en los comentarios del código:

1. `background` **vs** `background-color` al retintar: el atajo borra la capa de imagen.
2. `<svg>` **con** `width: auto` colapsa dentro de flex.
3. Los puntos del gráfico van **fuera** del SVG: dentro heredan el escalado no uniforme y se deforman.
4. `localStorage` **no aguanta**: 3,7 MB de datos con 40 canchas y el techo son ~4 MB. `setItem` lanza y se pierde el negocio entero. Por eso IndexedDB en la demo, y Postgres en producción.
5. `indexedDB.open` **puede no responder nunca** (borrado en cola): sin un temporizador de 3 s, el arranque no termina y el visitante ve una pantalla en blanco.
6. **Sin build no hay hash en los nombres de archivo**: una foto reemplazada con el mismo nombre se queda en caché, y un 404 antiguo también. De ahí `ASSET_V`. Al migrar a Next.js esto **desaparece** — lo resuelve el bundler.
7. **La generación de la semilla debe ser lineal**: comprobar duplicados sobre un array creciente la volvía cuadrática (10,5 s con 40 canchas, hilo congelado y sin indicador).
8. **La conversación de Neo encadena** `setTimeout` **durante ~20 s**: hay que cortarla al cambiar de vista o sigue corriendo sobre nodos desprendidos.

12. Plan de construcción

Cinco fases. Cada una termina en algo que se puede enseñar y que alguien puede usar. **Ninguna fase se cierra sin sus criterios de aceptación.**

Fase 0 · Cimientos (1 semana)

- Next.js + TypeScript + Tailwind con los tokens mapeados.
- Postgres con el esquema de §5.2 y RLS de §5.4.
- Auth: dueño con correo/contraseña, jugador con OTP por teléfono.
- Layout multi-tenant `/[complejo]` con navegación por rol.

Aceptación: dos complejos en la misma base; el dueño de uno no ve ni una fila del otro, comprobado con una consulta directa contra RLS.

Fase 1 · La agenda (2 semanas) — *lo mínimo vendible*

- Onboarding de cuatro preguntas escribiendo en base de datos.
- Canchas: crear, renombrar, tarifar, activar, foto.
- Reservas: agenda hora × cancha, 14 días, crear, abonar, cancelar (R19).
- Panel: facturado, cobrado, por cobrar, serie diaria, ranking por cancha.
- Realtime en la agenda.

Aceptación: un complejo real opera una semana entera sin cuaderno. Dos personas creando reservas a la vez en la misma franja: una gana, la otra ve un mensaje claro, nunca hay dos (R4).

Fase 2 · El jugador (2 semanas)

- Registro por teléfono y atribución de reservas anteriores (§7.1).
- Reservar: deporte, día, cancha, hora, forma de pago.
- Inicio, Mis reservas, Mi histórico.

Aceptación: un jugador reserva desde el celular y el dueño lo ve aparecer en la agenda sin recargar. El momento de §1.3, ahora con dos personas y dos dispositivos.

Fase 3 · Dinero de verdad (1–2 semanas)

- Pasarela para el adelanto, con bloqueo de 15 min (P2) y webhook idempotente (P3, P4).
- Tabla `pagos`, devoluciones, registro de efectivo en mostrador.
- Suscripción y estados de plan.

Aceptación: cien pagos de prueba, incluidos webhooks duplicados y pagos que nunca llegan; ni un cobro doble ni una franja bloqueada de más.

Fase 4 · Comunidad y Neo (2–3 semanas)

- Equipos, plantillas, torneos, llaves, tablas, salón de la fama.
- WhatsApp Cloud API, plantillas aprobadas, recordatorios.
- Escalado a LLM con herramientas acotadas y traspaso a humano.

Aceptación: un torneo de 8 equipos jugado de principio a fin corrigiendo un marcador a mitad: todas las estadísticas cuadran sin tocar nada más (R31). Veinte conversaciones de WhatsApp reales terminan en reserva sin intervención.

Lo que se queda como está

La **demo pública** sigue viva en `/demo`, sin cuenta, con el conmutador de rol. Es la herramienta de venta y no se toca salvo para arreglar lo que se rompa.

13. Medición

13.1 Del negocio del cliente

Ocupación por cancha y franja · facturado vs cobrado · saldo promedio · reservas por origen (cuántas atiende Neo sin humano) · cancelaciones y no-shows · horas muertas recuperadas.

13.2 De la demo comercial

Ya implementado en `app/core/lead.js` y se conserva:

- **El perfil comercial completo del prospecto sin formulario:** qué deportes tiene, cuántas canchas, a qué precio. Lo entrega en el onboarding porque le sirve, no porque se lo pidamos.
- **Señales de enganche:** minutos, secciones distintas abiertas, si habló con el asistente, si volvió.
- **El contacto se pide DENTRO de la demo, con su para qué a la vista:** el WhatsApp al que Neo le mandaría las reservas, y la demo se lo enseña funcionando con su propio número.
- Nunca bloquea ni rompe: si el CRM está caído, la demo sigue igual.

14. Decisiones abiertas

Cosas que hay que decidir antes de que el código las decida solo:

1. **Duración variable de reserva.** Hoy es una hora exacta (R1). Pádel y tenis a menudo se juegan 90 minutos. Cambiarlo toca la agenda, la disponibilidad y Neo. *Recomendación: fase 5, con franjas de 30 min como unidad interna y bloques de 60/90 como producto.*
2. **Precios por franja.** Hora pico vale más que un martes a las 10 a. m. Toda la estructura lo aguanta (total está congelado en la reserva, R8), pero hay que decidir la interfaz.
3. **Multi-sede.** Hoy un dueño, un complejo. El índice único que lo impone está señalado en la migración actual y se quita el día que haga falta.
4. **Quién cobra.** §8.2 asume que el dinero va a la cuenta del complejo. Ser intermediarios de fondos cambia el régimen regulatorio del negocio.
5. **Número de WhatsApp:** uno por complejo (mejor experiencia, más fricción de alta) o uno compartido (alta inmediata, mensaje menos personal).
6. **Deportes fuera de los cinco.** El modelo lo soporta; la iconografía y los planos de cancha están dibujados a mano y hay que dibujarlos.

15. Anexos

15.1 Glosario

TÉRMINO	QUÉ ES
Complejo	El negocio. Un tenant.
Cancha	El recurso que se reserva.
Franja	Una hora concreta de una cancha en un día.
Reserva	Una franja tomada por alguien.
Adelanto	Lo que se paga para apartar (50%, R10).
Saldo	Lo que falta por pagar.
Facturado / Cobrado	Lo que vale / lo que entró (R12, R13).
Cupo	Una plaza de equipo en un torneo.
Neo AI	El asistente de WhatsApp.
El momento	Reservar como jugador y verlo caer en la agenda del dueño.

15.2 Archivos de referencia en este repositorio

ARCHIVO	QUÉ CONTIENE
README.md	Qué hace cada sección, en prosa
DESIGN.md	El sistema de diseño y el porqué de cada valor
docs/MOLDE.md	Qué se hereda de esta demo a la siguiente
docs/PLAN.md	Las decisiones de la reconstrucción
docs/DESPLIEGUE.md	Cómo se despliega
BRIEF-CURSOR.md	Brief corto para pegar en un agente de código
themes/momentum.css	Los tokens: la fuente de verdad del diseño
supabase/migrations/0001_espacios.sql	El esquema actual, con RLS y concurrencia optimista

15.3 Cómo correr la demo

```
cd Sportplatz && python3 dev-server.py # http://localhost:5173
```

Son módulos ES: **necesita servirse por HTTP**. `dev-server.py` manda `no-store`; con el `http.server` normal el navegador se queda con el JS viejo después de cada edición y parece que los cambios no entraron.

Sportplatz · Fily & Co · documento maestro, 2026-08-20.